Banking Transaction & Account Management System

Project Milestone Report

Course: MTM4692 – Applied SQL

Team Members:

- Aykut ANLAK 22058606
- Gökay BAHTİYAR 21058048
- Umut Berke AYGÜNDÜZ 21058019
- Uras AMAN 21052064
- Yusuf Doğan GÜNEŞ 22052008

Instructor: Fettah KIRAN

1. Introduction

This project proposes a relational database system called **Banking Transaction & Account Management System**. The purpose of the system is to manage core banking data in a structured and consistent way using SQLite.

The project focuses on two main goals: **operational tracking** and **reporting**. In banking environments, large volumes of interconnected data are generated through customers, accounts, transactions, cards, loans, and payments. A relational database makes it possible to organize this data efficiently and to perform meaningful SQL-based analysis.

2. Problem Statement

One of the key challenges in banking systems is the difficulty of **tracking operational activities and producing accurate reports** from interconnected financial data.

Banks handle many different types of records at the same time, including:

- customer information
- account ownership
- branch assignments
- transaction history
- card records
- loan details
- payment activity

If this data is not stored in a well-designed relational structure, it becomes difficult to:

- monitor account activity
- track transaction flows
- compare branch performance
- identify active or unusual accounts
- produce reliable summaries for reporting

This project addresses that problem by designing a relational database that stores banking entities in a structured way and supports analytical SQL queries for operational monitoring and reporting.

3. Why It Matters

Operational tracking and reporting are essential in financial systems. Banks depend on accurate and well-structured data in order to monitor activities, evaluate performance, and support decision-making.

This project matters because a properly designed database can:

- improve data consistency
- reduce redundancy
- simplify operational monitoring
- support faster and clearer reporting
- provide a foundation for future analytical or anomaly-detection queries

For example, a bank may need to answer questions such as:

- Which customers generate the highest transaction volume?
- Which branches are the most active?
- Which accounts show above-average activity?
- How are transaction types distributed across channels?

A relational database is a strong fit for this problem because banking data is highly structured and depends on clear relationships and integrity constraints.

---

4. Project Scope

The project is intentionally limited to a realistic and manageable scope suitable for a milestone submission. It does not aim to build a full banking application interface. Instead, it focuses on the **database layer**.

The system includes the following seven entities:

- **customers**
- **branches**
- **accounts**
- **transactions**
- **cards**
- **loans**
- **payments**

This scope is large enough to demonstrate relational modeling and SQL features, while still being feasible to implement within the course timeline. The project milestone is expected to include a project proposal, schema sketch, and sample SQL features rather than a complete final system.

---

5. Preliminary Schema Design

The proposed schema is based on seven interconnected tables.

5.1 Customers

The `customers` table stores personal and contact information about bank customers, including name, national ID, email, phone, city, and registration date.

5.2 Branches

The `branches` table stores basic information about bank branches, including branch name, city, and address.

5.3 Accounts

The `accounts` table stores bank account information and connects customers to branches. It includes account number, account type, balance, currency, status, and opening date.

This is the **central table** in the system, because most operational records are connected to accounts.

5.4 Transactions

The `transactions` table stores financial movements such as deposits, withdrawals, transfers, and payments. It includes amount, transaction type, channel, branch, status, and date.

The field `related_account_id` is used to represent the counterparty account in transfer operations.

5.5 Cards

The `cards` table stores debit and credit card records linked to accounts.

5.6 Loans

The `loans` table stores customer loan information, including loan type, principal amount, interest rate, dates, and status.

5.7 Payments

The `payments` table stores payment activities such as bill payments, loan payments, and card payments linked to accounts.

---

6. ERD Explanation

The Entity Relationship Diagram (ERD) of the project consists of the seven core tables listed above. The diagram shows how the banking entities are connected through primary and foreign keys.

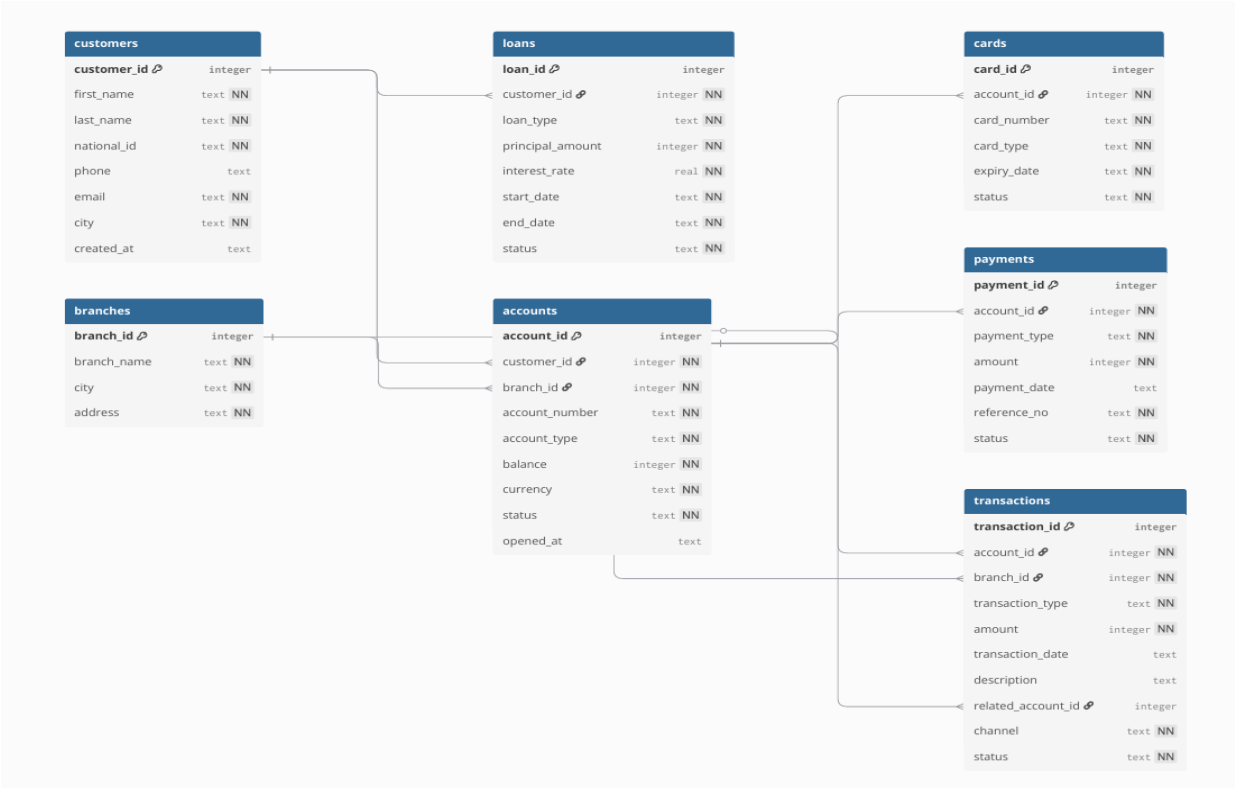
The main relationships are:

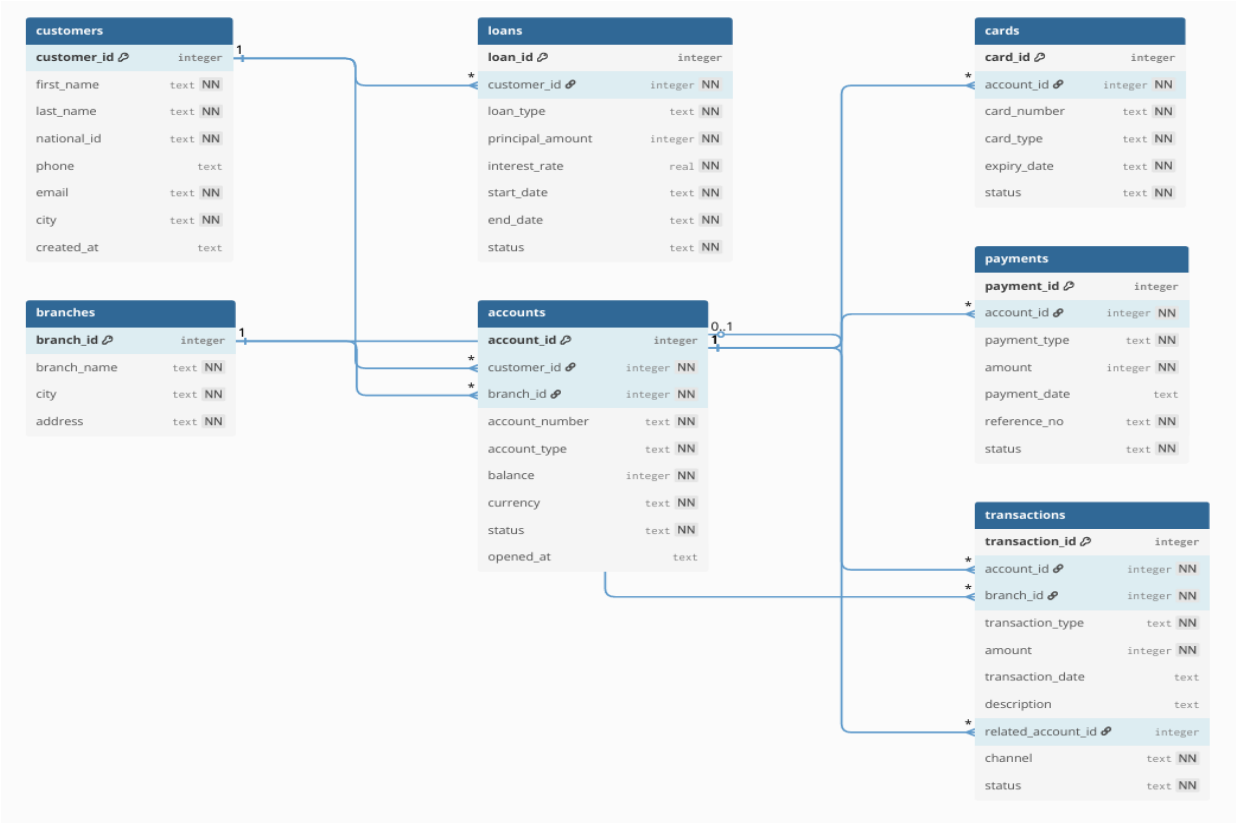
- one customer can have many accounts
- one branch can manage many accounts
- one account can have many transactions
- one account can have many cards
- one customer can have many loans
- one account can have many payments

The `accounts` table plays the central role in the design, because transactions, cards, and payments all depend on accounts. This makes the schema suitable for both operational tracking and reporting.

[Figure 1: Entity Relationship Diagram]

Figure 1 shows the Entity Relationship Diagram (ERD) of the proposed banking system. The diagram illustrates the relationships between the main entities such as customers, accounts, transactions, branches, cards, loans, and payments. The accounts table is the central entity, connecting most operational data, including transactions, cards, and payments. The relationships are primarily one-to-many, ensuring a scalable and normalized database design.





---

7. Design Decisions

7.1 Monetary Values Stored as INTEGER (Cents)

All monetary fields such as:

- account balance
- transaction amount
- principal loan amount
- payment amount

are stored as **INTEGER values representing cents**.

For example:

- 150.75 TRY is stored as `15075`

This decision avoids floating-point precision problems and follows a common best practice in financial database design. SQLite course material also emphasizes storing money carefully and suggests integer-based storage as a good practice for precise financial values.

7.2 Data Integrity

The schema uses:

- **PRIMARY KEY**
- **FOREIGN KEY**
- **UNIQUE**
- **NOT NULL**
- **CHECK**
- **DEFAULT**

constraints in order to preserve consistency and prevent invalid data.

7.3 Separation of Transactions and Payments

Transactions and payments are modeled separately.

- `transactions` represent raw financial movements
- `payments` represent higher-level payment actions such as bills, loan payments, or card-related payments

This separation improves reporting clarity and supports more flexible analysis.

7.4 Card Design

The `cards` table does not include CVV. Since CVV is not needed for the project queries and is sensitive in real-world systems, it was intentionally excluded from the final schema.

---

8. Synthetic Data Strategy

The project uses **synthetic data** instead of a public real-world dataset. This approach was chosen because it provides better control over the schema and ensures that the generated records are fully compatible with the designed tables and relationships.

The synthetic data is designed to be realistic and includes:

- 8 branches
- 30 customers
- 45 accounts
- 160 transactions
- 20 cards
- 15 loans
- 30 payments

Using synthetic data also makes it easier to generate scenarios that support the intended analytical queries, such as above-average activity and branch-level summaries.

Data is inserted through SQL scripts, and the full project structure includes:

- `schema.sql`
- `insert.sql`
- `queries.sql`

---

9. Proposed SQL Features

The project demonstrates several SQL techniques covered in the course.

9.1 DDL

The schema is created using `CREATE TABLE` statements with appropriate constraints.

9.2 DML

Data is inserted into the tables through SQL insert scripts.

9.3 Multi-table JOINs

The project uses joins to combine customers, accounts, branches, and transactions.

9.4 Aggregation and GROUP BY

The system supports analytical summaries using `COUNT`, `SUM`, and `GROUP BY`.

9.5 HAVING and Subqueries

The project includes filtering of grouped results and nested queries for identifying above-average activity.

These SQL features are explicitly expected in the project proposal and rubric, which mentions views, joins, and aggregation as important parts of the milestone and final project expectations.

---

10. Sample Queries

10.1 Customer-Based Total Transaction Volume

This query shows the total number and volume of transactions performed by each customer across all accounts.

```
SELECT

   c.customer_id,

   c.first_name || ' ' || c.last_name AS customer_name,

   COUNT(t.transaction_id) AS transaction_count,

   SUM(t.amount) AS total_transaction_cents,

   PRINTF('%.2f', SUM(t.amount) / 100.0) AS total_transaction_try

FROM customers c

JOIN accounts a

   ON c.customer_id = a.customer_id

JOIN transactions t

   ON a.account_id = t.account_id

GROUP BY c.customer_id, customer_name

ORDER BY total_transaction_cents DESC;
```

customer_id	customer_name	transaction_count	total_transaction_cents	total_transaction_try
15	Burak Aksoy	7	82000	820.00
14	Merve Tas	7	81125	811.25
13	Ibrahim Aslan	7	80250	802.50
12	Hatice Kilic	7	79375	793.75
11	Hasan Ozdemir	7	78500	785.00
10	Elif Kurt	7	77625	776.25
9	Huseyin Dogan	7	76750	767.50
8	Emine Arslan	7	75875	758.75
7	Mustafa Aydin	7	75000	750.00
6	Zeynep Yildiz	7	74125	741.25
5	Ahmet Sahin	7	73250	732.50
4	Fatma Celik	7	72375	723.75
3	Mehmet Kaya	7	71500	715.00
2	Ayse Demir	7	70625	706.25
1	Ali Yilmaz	7	69750	697.50
25	Serkan Bozkurt	4	50250	502.50
24	Pelin Kara	4	49750	497.50
23	Tolga Gunes	4	49250	492.50
22	Derya Acar	4	48750	487.50
21	Omer Koc	4	48250	482.50
20	Ceren Kaplan	4	47750	477.50
19	Onur Eren	4	47250	472.50
18	Gizem Bulut	4	46750	467.50
17	Kerem Can	4	46250	462.50
16	Seda Yavuz	4	45750	457.50
30	Sinem Ucar	3	31125	311.25
29	Deniz Polat	3	30750	307.50
28	Ece Akin	3	30375	303.75
27	Yusuf Tekin	3	30000	300.00
26	Buse Altun	3	29625	296.25

10.2 Branch-Based Account and Transaction Summary

This query compares branches in terms of account count, transaction count, and total transaction volume.

```
SELECT

  b.branch_id,

  b.branch_name,

  COUNT(DISTINCT a.account_id) AS total_accounts,

  COUNT(t.transaction_id) AS total_transactions,

  COALESCE(SUM(t.amount), 0) AS total_volume_cents,

  PRINTF('%.2f', COALESCE(SUM(t.amount), 0) / 100.0) AS total_volume_try
```

```
FROM branches b

LEFT JOIN accounts a

   ON b.branch_id = a.branch_id

LEFT JOIN transactions t

   ON b.branch_id = t.branch_id

GROUP BY b.branch_id, b.branch_name

ORDER BY total_volume_cents DESC;
```

branch_id	branch_name	total_accounts	total_transactions	total_volume_cents	total_volume_try
5	Bursa Subesi	6	120	1335000	13350.00
4	Izmir Merkez	6	120	1320000	13200.00
3	Ankara Merkez	6	120	1305000	13050.00
2	Besiktas Subesi	6	120	1290000	12900.00
1	Istanbul Merkez	6	120	1275000	12750.00
8	Eskisehir Subesi	5	100	1150000	11500.00
7	Trabzon Subesi	5	100	1137500	11375.00
6	Antalya Subesi	5	100	1125000	11250.00

10.3 Accounts With Above-Average Total Transaction Volume

This query finds accounts whose total transaction amount is above the average total transaction amount per account.

```
SELECT

   a.account_id,

   a.account_number,

   COUNT(t.transaction_id) AS transaction_count,

   SUM(t.amount) AS total_amount_cents,

   PRINTF('%.2f', SUM(t.amount) / 100.0) AS total_amount_try

FROM accounts a

JOIN transactions t

   ON a.account_id = t.account_id

GROUP BY a.account_id, a.account_number
```

```
HAVING SUM(t.amount) > (
    SELECT AVG(account_total)
    FROM (
        SELECT SUM(amount) AS account_total
        FROM transactions
        GROUP BY account_id
    ) AS sub
)
ORDER BY total_amount_cents DESC;
```

account_id	account_number	transaction_count	total_amount_cents	total_amount_try
25	TR000025	4	50250	502.50
24	TR000024	4	49750	497.50
23	TR000023	4	49250	492.50
22	TR000022	4	48750	487.50
21	TR000021	4	48250	482.50
20	TR000020	4	47750	477.50
19	TR000019	4	47250	472.50
18	TR000018	4	46750	467.50
17	TR000017	4	46250	462.50
16	TR000016	4	45750	457.50
15	TR000015	4	45250	452.50
14	TR000014	4	44750	447.50
13	TR000013	4	44250	442.50
12	TR000012	4	43750	437.50
11	TR000011	4	43250	432.50
10	TR000010	4	42750	427.50
9	TR000009	4	42250	422.50
8	TR000008	4	41750	417.50
7	TR000007	4	41250	412.50
6	TR000006	4	40750	407.50
5	TR000005	4	40250	402.50
4	TR000004	4	39750	397.50

These three queries were selected because they directly support the project theme of operational tracking and reporting.

11. Repository and File Structure

All project files are prepared to be stored in a GitHub repository, as required by the course project instructions. The milestone explicitly asks for a GitHub repository, technical report, schema sketch, and sample queries.

Suggested structure:

/sql/schema.sql

/sql/insert.sql

/sql/queries.sql

/images/erd.png

/docs/report.pdf

README.md

12. Conclusion

This project proposes a relational banking database focused on **operational tracking and reporting**. The schema includes seven well-connected entities and supports realistic banking analysis through structured SQL queries.

The design ensures:

- data consistency
- relational integrity
- analytical flexibility
- practical reporting capability

As a milestone submission, this proposal establishes a solid foundation for the final implementation. The system is realistic, feasible, and directly aligned with the SQL concepts covered in the course.

GitHub Repository

GitHub Repository Link: https://github.com/aygunduzumutberke/banking-database-project